

Contoh Stored Procedure pada MongoDB

1. DBMS yang Digunakan

Aplikasi ini menggunakan MongoDB 7 sebagai database. MongoDB tidak memiliki stored procedure native seperti `CREATE PROCEDURE` pada MySQL atau PostgreSQL. MongoDB menggunakan query document, aggregation pipeline, dan transaksi.

Karena itu, implementasi yang paling sesuai untuk kebutuhan ini adalah procedure-like operation pada application layer PHP. Operasi database tetap diletakkan di repository, sedangkan script PHP menjadi entry point yang dapat dijalankan berulang kali.

2. Kebutuhan

Collection yang digunakan:

```
database: app_db
collection: products
```

Struktur dokumen:

```
{
  "id": 1,
  "name": "Product 1",
  "price": 10.99
}
```

3. Procedure yang Diimplementasikan

Nama operasi: `getProductsByMinimumPrice`

Tujuan:

- mengambil produk dengan harga minimal tertentu;
- menyembunyikan field internal MongoDB `_id`;
- mengurutkan produk dari harga tertinggi;
- mengembalikan hasil dalam format JSON.

Implementasi ini menggunakan aggregation pipeline MongoDB dengan tahap:

1. `$match` untuk filter harga;
2. `$project` untuk memilih field output;
3. `$sort` untuk mengurutkan hasil.

4. Script PHP Lengkap

File implementasi: `slim-framework/database/product_procedure.php`

```
<?php

declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$minimumPrice = isset($argv[1]) ? filter_var($argv[1],
FILTER_VALIDATE_FLOAT) : 0.0;

if ($minimumPrice === false || $minimumPrice < 0) {
    fwrite(STDERR, "Minimum price must be a non-negative number.\n");
    exit(1);
}

$client = new MongoDB\Client((string) getenv('MONGODB_URI'));
$databaseName = getenv('MONGODB_DATABASE') ?: 'app_db';
$collection = $client->selectCollection($databaseName, 'products');

$pipeline = [
    [
        '$match' => [
            'price' => ['$gte' => $minimumPrice],
        ],
    ],
    [
        '$project' => [
            '_id' => 0,
            'id' => 1,
            'name' => 1,
            'price' => 1,
        ],
    ],
    [
        '$sort' => ['price' => -1],
    ],
];

$products = iterator_to_array($collection->aggregate($pipeline), false);
$result = [];

foreach ($products as $product) {
    $result[] = [
        'id' => (int) $product['id'],
        'name' => (string) $product['name'],
        'price' => (float) $product['price'],
    ];
}

echo json_encode(
    [
        'minimum_price' => $minimumPrice,
```

```
        'total' => count($result),  
        'products' => $result,  
    ],  
    JSON_PRETTY_PRINT  
) . PHP_EOL;
```

5. Cara Menjalankan

Pastikan container sudah aktif:

```
docker compose up -d mongodb php
```

Jalankan procedure dengan harga minimum 10:

```
docker compose exec php php database/product_procedure.php 10
```

6. Hasil Implementasi

Berdasarkan data seed yang tersedia, hasilnya adalah:

```
{  
  "minimum_price": 10,  
  "total": 2,  
  "products": [  
    {  
      "id": 2,  
      "name": "Product 2",  
      "price": 19.99  
    },  
    {  
      "id": 1,  
      "name": "Product 1",  
      "price": 10.99  
    }  
  ]  
}
```

7. Versi MongoDB Shell

Aggregation yang digunakan oleh script PHP dapat diuji langsung dengan **mongosh**:

```
use app_db;  
  
db.products.aggregate([
```

```
{
  $match: {
    price: { $gte: 10 }
  }
},
{
  $project: {
    _id: 0,
    id: 1,
    name: 1,
    price: 1
  }
},
{
  $sort: { price: -1 }
}
]);
```

8. Kesimpulan

MongoDB tidak menyediakan stored procedure permanen seperti database relasional. Oleh sebab itu, implementasi procedure pada aplikasi ini dibuat sebagai operasi PHP yang menggunakan aggregation pipeline MongoDB. Repository dan service CRUD yang sudah ada tetap mengikuti prinsip clean architecture, sehingga operasi ini dapat dipindahkan ke use case terpisah tanpa mengubah controller HTTP.